

GUIDE D'IMPLÉMENTATION RAG - CHATBOT ANDF BÉNIN

Vue d'ensemble

Ce guide vous accompagne dans l'implémentation complète d'un système RAG (Retrieval-Augmented Generation) pour le chatbot de l'Agence Nationale du Domaine et du Foncier du Bénin.

ARCHITECTURE TECHNIQUE RECOMMANDÉE

Stack technologique

Frontend: React.js + TypeScript
Backend: Python FastAPI + Uvicorn
LLM: Llama 3.1-8B ou Mistral-7B-Instruct
Embeddings: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Vector DB: ChromaDB (production) ou Qdrant
Cache: Redis
Database: PostgreSQL (métadonnées)

Architecture RAG Pipeline

User Query → Intent Detection → Entity Recognition → Vector Search →
Context Retrieval → LLM Generation → Response Validation → Output

PHASE 1 : PRÉPARATION DES DONNÉES

1.1 Structure des documents

```
python
```

```
documents = {  
  "procedures": [  
    {  
      "id": "proc_001",  
      "title": "Confirmation de droits fonciers",  
      "content": "Procédure pour obtenir un titre foncier...",  
      "category": "procedure",  
      "cost": 100000,  
      "duration": "120 jours",  
      "documents_required": [...],  
      "steps": [...],  
      "keywords": ["titre", "foncier", "confirmation", "propriété"]  
    }  
  ],  
  "tariffs": [...],  
  "communes": [...],  
  "definitions": [...]  
}
```

1.2 Chunking Strategy

python

```
def chunk_documents(documents, chunk_size=500, overlap=50):
    """
    Découpe intelligente des documents par sections logiques
    """
    chunks = []
    for doc in documents:
        if doc["category"] == "procedure":
            # Chunk par étape de procédure
            for i, step in enumerate(doc["steps"]):
                chunks.append({
                    "id": f"{doc['id']}_step_{i}",
                    "content": f"Procédure {doc['title']} - Étape {i+1}: {step}",
                    "metadata": {
                        "procedure": doc["title"],
                        "step_number": i+1,
                        "cost": doc["cost"],
                        "category": "procedure_step"
                    }
                })
        elif doc["category"] == "definition":
            # Chunks complets pour définitions
            chunks.append({
                "id": doc["id"],
                "content": f"Définition {doc['term']}: {doc['definition']}",
                "metadata": {
                    "term": doc["term"],
                    "category": "definition"
                }
            })
    return chunks
```

PHASE 2 : CONFIGURATION EMBEDDINGS

2.1 Modèle d'embeddings multilingue

python

```

from sentence_transformers import SentenceTransformer

# Modèle recommandé pour français + langues locales
model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

def create_embeddings(texts):
    """Génère des embeddings pour les textes"""
    return model.encode(texts, convert_to_tensor=True)

```

2.2 Configuration ChromaDB

```

python

import chromadb
from chromadb.config import Settings

# Configuration pour production
client = chromadb.Client(Settings(
    chroma_db_impl="duckdb+parquet",
    persist_directory="./chroma_db"
))

# Création de collections spécialisées
procedures_collection = client.create_collection(
    name="andf_procedures",
    metadata={"description": "Procédures foncières ANDF"}
)

definitions_collection = client.create_collection(
    name="andf_definitions",
    metadata={"description": "Définitions juridiques"}
)

tariffs_collection = client.create_collection(
    name="andf_tariffs",
    metadata={"description": "Barèmes et coûts"}
)

```

PHASE 3 : SYSTÈME DE RECONNAISSANCE D'ENTITÉS

3.1 Entités spécifiques au foncier béninois

```

python

```

```

import re
from typing import Dict, List

class ANDFEntityRecognizer:
    def __init__(self):
        self.commune_patterns = self.load_commune_patterns()
        self.procedure_patterns = self.load_procedure_patterns()
        self.document_patterns = self.load_document_patterns()

    def extract_entities(self, text: str) -> Dict[str, List[str]]:
        """Extrait les entités spécifiques du domaine foncier"""
        entities = {
            "communes": [],
            "procedures": [],
            "documents": [],
            "montants": []
        }

        # Reconnaissance des communes
        for commune in self.commune_patterns:
            if commune.lower() in text.lower():
                entities["communes"].append(commune)

        # Reconnaissance des procédures
        for proc_pattern in self.procedure_patterns:
            if re.search(proc_pattern, text, re.IGNORECASE):
                entities["procedures"].append(proc_pattern)

        # Reconnaissance des montants en F CFA
        montant_pattern = r'(\d+(?:\.\d+)?)\s*(?:f\s*cfa|fcfa|francs?)'
        montants = re.findall(montant_pattern, text, re.IGNORECASE)
        entities["montants"] = [float(m) for m in montants]

        return entities

    def load_commune_patterns(self):
        return [
            "Cotonou", "Porto-Novo", "Parakou", "Abomey-Calavi",
            "Allada", "Ouidah", "Lokossa", "Natitingou", # ... toutes les 77 communes
        ]

    def load_procedure_patterns(self):
        return [
            r"titre\s+foncier", r"certificat\s+d['\"]appartenance",
            r"confirmation\s+de\s+droits?", r"mutation",
        ]

```

```
r"hypoth[ée]que", r"morcellement", r"bornage"  
]
```

3.2 Détection d'intention

python

```
class IntentClassifier:  
    def __init__(self):  
        self.intents = {  
            "procedure_info": [  
                r"comment faire", r"proc[ée]dure", r"étapes?", r"comment obtenir"  
            ],  
            "cost_inquiry": [  
                r"co[û]t", r"prix", r"tarif", r"frs", r"combien"  
            ],  
            "document_required": [  
                r"documents?", r"pi[ée]ces?", r"faut.*quoi", r"besoin de quoi"  
            ],  
            "location_info": [  
                r"o[ù]", r"adresse", r"bureau", r"bcdf", r"contact"  
            ],  
            "duration_info": [  
                r"dur[ée]e", r"temps", r"d[ée]lai", r"combien de temps"  
            ]  
        }  
  
    def classify_intent(self, text: str) -> str:  
        for intent, patterns in self.intents.items():  
            for pattern in patterns:  
                if re.search(pattern, text, re.IGNORECASE):  
                    return intent  
        return "general_info"
```

PHASE 4 : RETRIEVAL HYBRIDE

4.1 Recherche vectorielle + BM25

python

```

from rank_bm25 import BM25Okapi
import numpy as np

class HybridRetriever:
    def __init__(self, vector_db, documents):
        self.vector_db = vector_db
        self.documents = documents
        self.setup_bm25()

    def setup_bm25(self):
        corpus = [doc["content"].split() for doc in self.documents]
        self.bm25 = BM25Okapi(corpus)

    def hybrid_search(self, query: str, top_k: int = 5) -> List[Dict]:
        # Recherche vectorielle
        vector_results = self.vector_db.query(
            query_texts=[query],
            n_results=top_k*2
        )

        # Recherche BM25
        query_tokens = query.split()
        bm25_scores = self.bm25.get_scores(query_tokens)
        bm25_top = np.argsort(bm25_scores)[-top_k*2:][::-1]

        # Combinaison et re-ranking
        combined_results = self.combine_and_rerank(
            vector_results, bm25_top, query
        )

        return combined_results[:top_k]

    def combine_and_rerank(self, vector_results, bm25_results, query):
        # Logique de combinaison et re-ranking
        # Poids: 70% vector, 30% BM25
        pass

```

4.2 Filtrage contextuel

python

```
def contextual_filter(results: List[Dict], entities: Dict, intent: str) -> List[Dict]:
    """Filtre les résultats selon le contexte détecté"""
    filtered = []

    for result in results:
        metadata = result.get("metadata", {})

        # Filtrage par commune
        if entities.get("communes") and metadata.get("commune"):
            if metadata["commune"] not in entities["communes"]:
                continue

        # Filtrage par type de procédure
        if entities.get("procedures") and metadata.get("procedure"):
            if not any(proc in metadata["procedure"].lower()
                        for proc in entities["procedures"]):
                continue

        # Filtrage par intention
        if intent == "cost_inquiry" and "cost" not in metadata:
            continue

        filtered.append(result)

    return filtered
```

PHASE 5 : GÉNÉRATION DE RÉPONSES

5.1 Templates de prompts spécialisés

python

PROMPT_TEMPLATES = {

"procedure_info": ""

Tu es un expert en droit foncier béninois travaillant pour l'ANDF.

Un citoyen te demande des informations sur une procédure.

CONTEXTE: {context}

ENTITÉS DÉTECTÉES: {entities}

QUESTION: {question}

Fournis une réponse claire et structurée avec:

1. Les étapes de la procédure
2. Les documents requis
3. Le coût total
4. La durée
5. Le service compétent

Utilise un langage simple et accessible.

""

"cost_inquiry": ""

Tu es un conseiller tarifaire de l'ANDF.

Calcule et explique les coûts fonciers.

CONTEXTE: {context}

QUESTION: {question}

Fournis:

1. Le coût de base
2. Les frais variables
3. Le total estimé
4. Les modalités de paiement

Format: coûts en F CFA avec explications claires.

""

"general_info": ""

Tu es un agent d'accueil ANDF. Réponds de manière professionnelle

et redirige vers les services appropriés si nécessaire.

CONTEXTE: {context}

QUESTION: {question}

Réponds de manière utile et précise.

""

}

5.2 Générateur de réponses

python

```
from transformers import AutoTokenizer, AutoModelForCausalLM

class ANDFResponseGenerator:
    def __init__(self, model_name="mistralai/Mistral-7B-Instruct-v0.1"):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForCausalLM.from_pretrained(model_name)

    def generate_response(self, context: str, query: str, intent: str, entities: Dict) -> str:
        # Sélection du template approprié
        template = PROMPT_TEMPLATES.get(intent, PROMPT_TEMPLATES["general_info"])

        # Construction du prompt
        prompt = template.format(
            context=context,
            question=query,
            entities=entities
        )

        # Génération
        inputs = self.tokenizer(prompt, return_tensors="pt")
        outputs = self.model.generate(
            inputs.input_ids,
            max_length=1024,
            temperature=0.7,
            do_sample=True,
            pad_token_id=self.tokenizer.eos_token_id
        )

        response = self.tokenizer.decode(outputs[0], skip_special_tokens=True)
        return self.clean_response(response)

    def clean_response(self, response: str) -> str:
        """Nettoie et formate la réponse"""
        # Suppression du prompt original
        # Formatage
        # Validation
        return response
```

PHASE 6 : VALIDATION ET CONTRÔLE QUALITÉ

6.1 Système de validation des réponses

python

```
class ResponseValidator:
    def __init__(self):
        self.valid_costs = self.load_official_tariffs()
        self.valid_procedures = self.load_procedures()

    def validate_response(self, response: str, intent: str) -> Dict[str, bool]:
        validation = {
            "factual_accuracy": True,
            "cost_accuracy": True,
            "procedure_accuracy": True,
            "completeness": True
        }

        if intent == "cost_inquiry":
            validation["cost_accuracy"] = self.validate_costs(response)

        if intent == "procedure_info":
            validation["procedure_accuracy"] = self.validate_procedure(response)

        return validation

    def validate_costs(self, response: str) -> bool:
        """Vérifie que les coûts mentionnés sont corrects"""
        mentioned_costs = self.extract_costs_from_response(response)
        for cost in mentioned_costs:
            if cost not in self.valid_costs.values():
                return False
        return True
```

6.2 Fallback et escalade

python

```
def handle_low_confidence_response(confidence_score: float, query: str) -> str:
```

```
    """Gère les réponses avec faible confiance"""
```

```
    if confidence_score < 0.3:
```

```
        return f"""
```

```
Je ne suis pas sûr de pouvoir répondre précisément à votre question sur : "{query}"
```

```

Pour une réponse exacte, je vous recommande de :
```

```
1. Contacter directement votre BCDF
```

```
2. Appeler le +229 21 30 10 20
```

```
3. Visiter le site https://andf.bj
```

```

Puis-je vous aider avec autre chose ?
```

```
"""
```

```
    elif confidence_score < 0.6:
```

```
        return f"""
```

```
Voici les informations que j'ai trouvées, mais je vous conseille
de les vérifier auprès de votre BCDF local :
```

```
{response}
```

```

⚠ Information à confirmer auprès des services compétents.
```

```
"""
```

PHASE 7 : DÉPLOIEMENT ET MONITORING

7.1 API FastAPI

```
python
```

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="ANDF Chatbot API", version="1.0.0")

class ChatRequest(BaseModel):
    question: str
    user_location: str = None
    language: str = "fr"

class ChatResponse(BaseModel):
    answer: str
    confidence: float
    sources: List[str]
    suggested_actions: List[str]

@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    try:
        # Pipeline complet
        entities = entity_recognizer.extract_entities(request.question)
        intent = intent_classifier.classify_intent(request.question)
        context = hybrid_retriever.hybrid_search(request.question)
        response = response_generator.generate_response(
            context, request.question, intent, entities
        )

        return ChatResponse(
            answer=response,
            confidence=0.85,
            sources=["Code foncier 2013", "Procédures ANDF"],
            suggested_actions=["Contacter BCDF", "Préparer documents"]
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

7.2 Métriques et monitoring

python

```

import logging
from prometheus_client import Counter, Histogram

# Métriques
REQUEST_COUNT = Counter('andf_chatbot_requests_total', 'Total requests')
RESPONSE_TIME = Histogram('andf_chatbot_response_seconds', 'Response time')
CONFIDENCE_SCORE = Histogram('andf_chatbot_confidence', 'Confidence scores')

def log_interaction(query: str, response: str, confidence: float):
    """Log des interactions pour amélioration continue"""
    logging.info({
        "timestamp": datetime.now(),
        "query": query,
        "response_length": len(response),
        "confidence": confidence,
        "session_id": session.id
    })

```

PHASE 8 : AMÉLIORATION CONTINUE

8.1 Collecte de feedback

```

python

class FeedbackCollector:
    def __init__(self):
        self.db = PostgreSQL()

    def collect_feedback(self, session_id: str, rating: int, comment: str):
        """Collecte les retours utilisateurs"""
        self.db.execute("""
            INSERT INTO feedback (session_id, rating, comment, created_at)
            VALUES (%s, %s, %s, %s)
            """, (session_id, rating, comment, datetime.now()))

    def analyze_feedback(self) -> Dict:
        """Analyse des feedbacks pour amélioration"""
        pass

```

8.2 Mise à jour des données

```

python

```

```
def update_knowledge_base():  
    """Processus de mise à jour périodique"""  
    # 1. Scraping des nouvelles informations officielles  
    # 2. Validation des changements  
    # 3. Re-indexation vectorielle  
    # 4. Tests de régression  
    # 5. Déploiement graduel  
    pass
```

CHECKLIST DE DÉPLOIEMENT

✓ Pré-requis techniques

- ☐ Serveur avec GPU (minimum 8GB VRAM)
- ☐ Python 3.9+
- ☐ Dépendances installées
- ☐ Base vectorielle configurée
- ☐ Modèles téléchargés

✓ Tests de validation

- ☐ Tests unitaires (>90% couverture)
- ☐ Tests d'intégration
- ☐ Tests de charge
- ☐ Validation qualité réponses
- ☐ Tests multilingues

✓ Monitoring et alertes

- ☐ Métriques configurées
- ☐ Alertes seuils configurées
- ☐ Logs structurés
- ☐ Dashboard monitoring

✓ Sécurité

- ☐ Rate limiting
- ☐ Validation inputs
- ☐ Chiffrement données sensibles
- ☐ Audit trail

RESSOURCES COMPLÉMENTAIRES

Documentation officielle

- Site ANDF: <https://andf.bj>
- Code foncier: Version complète dans la base
- Contactez l'ANDF: +229 21 30 10 20

Support technique

- Documentation RAG: Voir guide technique
- Modèles recommandés: HuggingFace Hub
- Communauté: GitHub Issues

Guide d'implémentation v1.0 - Janvier 2025

Pour questions techniques: contact@andf-chatbot.bj